CellAsDistribution: Decision-Native Language Modeling of Single-Cell Gene Composition

Anonymous submission

Abstract

Language models of single cells write a cell as a sentence of gene names ordered by expression and generate it token by token, so their output is text that must be mapped back to counts before it can be scored. We propose instead to let a decision language model generate the cell directly: on a fixed gene panel, a cell's counts define a probability distribution over genes, and a softmax decision head on a fine-tuned language model returns this distribution from a text description of the cell's state in one forward pass. The output is the expected share of the cell's transcripts coming from each gene; it is normalized by construction and scored by the multinomial likelihood of held-out counts, on the same scale as every other method. On held-out CRISPRa perturbations in K562, the decoder is 0.10 nats per UMI better than an autoregressive Cell2Sentence model fine-tuned on the same backbone and budget, which adds no likelihood over the perturbed mean, and it decodes 90 to 540 times faster. Rewriting real cells as rank sentences shows that much of what a sentence loses lies in the map back to counts: the published reconstruction keeps 38% of the achievable likelihood gain, a map learned from training cells 88%; yet re-decoding the model's generated sentences with that learned map does not close the gap. In pre-registered controls, a decoder built from learned gene embeddings with the same softmax output matches the language model in likelihood, and a frozen language model is worse, so the gain comes from decoding the distribution directly; the fine-tuned language model contributes the text interface. Biologically, the decoder ties additive composition of known perturbation effects in likelihood and on the downstream differentially expressed genes with the targeted genes excluded, generates cells closer to real ones, and does not transfer to genes never perturbed in training. For cell types it reaches, but cannot exceed, the type mean on unseen donors, and it generates unseen types from their names, matching a name-matching baseline in likelihood while recovering the requested type far more often. All comparisons use leakage-controlled splits and matched control cells; new configurations were promoted by rules fixed before the test runs, and analyses added afterwards are labeled as such.

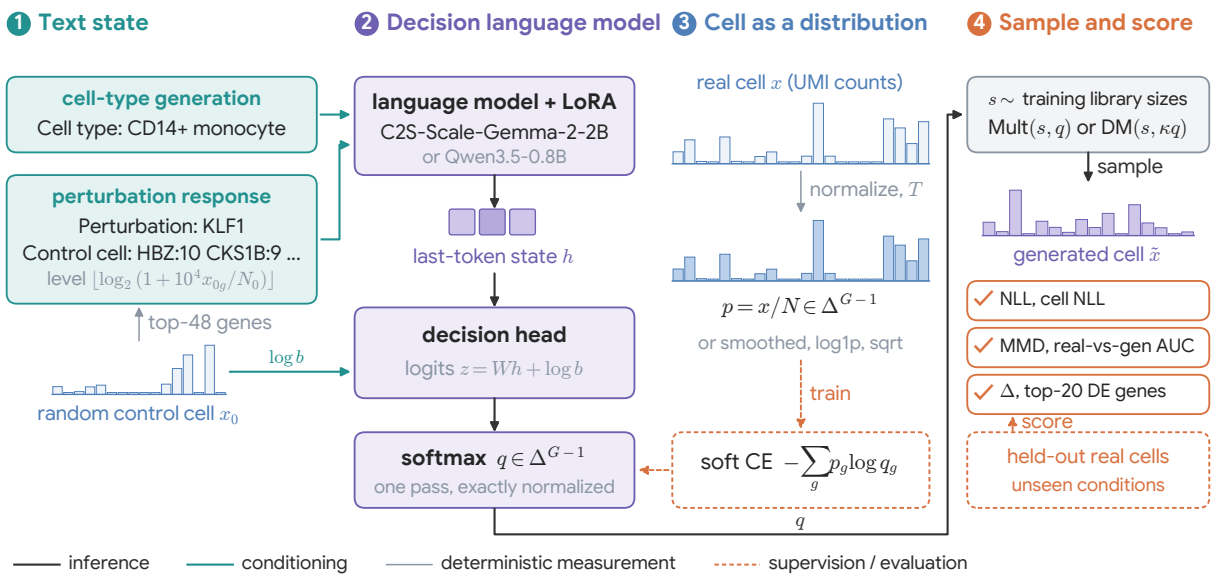


Figure 1: **A decision language model reads a text state and returns, in one forward pass, a distribution over a fixed gene panel; every method is scored with the same likelihood and sampler.** (1) The state is a prompt: a cell type, or a perturbation, optionally with a control cell x_0. (2) A LoRA-tuned language model encodes the prompt; a linear decision head on the last-token state, with an optional fixed log-prior b (none, the training mean, or the smoothed control cell), gives one logit per gene, and a softmax gives q on the simplex. (3) A real cell is the probability vector $p = x/N$ of its counts; training minimizes the cross-entropy between p and q. (4) A shared sampler draws cells from q; held-out cells of unseen donors, perturbations or cell types score the model.

1 Introduction

Predicting the cells of an unmeasured condition, a cell type in a new donor or a new genetic perturbation, is the central task of the "virtual cell" program [1]. A useful generator must say not only which genes change but how a cell's transcripts are distributed over genes, and it must be judged by the probability it assigns to cells it has not seen. Language models are attractive for this task because a condition can be stated in text: Cell2Sentence (C2S) writes each cell as a sentence of gene names ordered by decreasing expression and fine-tunes a pretrained language model on these sentences [2, 3]. The model then generates a cell token by token, and its output is text: a ranked list of names that must be mapped back to expression before it can be compared with counts.

We ask a simple question: why generate a sentence at all? On a fixed panel of G genes, the counts of a cell divided by their sum are a probability distribution over the panel, and together with the library size they reproduce the counts exactly (Observation 1). A *decision language model* answers a prompt not with text but with a probability over a fixed set of options in one forward pass [4, 5]. If the options are the genes of the panel, the answer *is* a cell: the expected share of its transcripts that each gene contributes. We call this framework CellAsDistribution (Figure 1). A text state (a cell type, or a perturbation with an optional control cell) goes in; a normalized gene distribution comes out; counts are sampled from it, and the multinomial likelihood of held-out counts scores it on the same scale as any other generator.

The idea is simple, so its value has to be measured, and we organize the paper around three questions. **(Q1) Interface.** Does generating the distribution directly beat generating a rank sentence, on the same backbone and budget, and where does the sentence lose? **(Q2) Language model.** Does the fine-tuned language model matter, or would any encoder of the condition do? **(Q3) Biology.** Does a better gene distribution mean better biology: the downstream response to a perturbation once the trivially induced target gene is set aside, the realism of single cells, and conditions never seen in training? We answer them on human PBMC (generation for unseen donors and unseen cell types) and on the CRISPRa screen of Norman et al. in K562 (response to held-out perturbations), where simple baselines are known to be hard to beat [6, 7]. Our contributions are:

1. **A decision-native generator of single cells.** A softmax decision head over a fixed gene panel turns a fine-

tuned language model into a one-pass generator whose output is a normalized gene composition with an exact multinomial score, comparable with count models and simple baselines (Section 3).

2. **A controlled comparison with sentence generation.** On the same backbone and budget, direct decoding gains 0.10 nats per UMI over autoregressive C2S at 90 to 540 times lower decoding cost; rewriting real cells shows that the rank sentence loses most of its likelihood in the map back to counts rather than in the ranks themselves; and matched controls separate the contribution of the language model (Sections 5.1 and 5.2).

3. **A biological boundary.** Under leakage-controlled splits, CellAsDistribution matches additive composition of known effects on likelihood and on downstream differentially expressed genes, generates more realistic cells, composes unseen cell types from their names, and fails where no method in our comparison succeeds: genes never perturbed in training (Sections 5.3–5.5).

2 Related work

Decision language models. A decision language model returns a probability over a bounded set of options for a typed question in a single pass instead of generating text to be parsed [5, 8]. The open implementation we build on attaches a classification head to the final hidden state of a pretrained model and trains it with the cross-entropy on soft targets [4]. The head itself is a softmax classifier; what is new here is the choice of options, the genes of a panel, which makes the answer a cell.

Cells as text and cell foundation models. Cell2Sentence fine-tunes language models on rank-ordered gene sentences and has been scaled to billions of parameters [2, 3]; its case for the representation is that expression is recovered from rank by a log-linear fit with high R^2. Other foundation models pretrain on expression tokens (scGPT, Geneformer) [9, 10], learn cell embeddings (UCE) [11], or decode the whole profile from a bottleneck token (BMFM-RNA) [12]; none returns a normalized distribution from a text condition.

Count generators and perturbation predictors. scVI and scANVI decode gene frequencies inside a negative-binomial likelihood conditioned on latent states and labels [13, 14]; scDiffusion samples cells with a conditional diffusion model [15]; the Gene Mover's Distance compares cells as distributions over genes [16]. Perturbation predictors such as scGen, CPA, GEARS and State map a perturbation to an expression change [17–20], and population models learn copulas, flows or transport between control and perturbed cells [21–26]. Independent benchmarks find that mean, additive and linear baselines are hard to beat [6, 7] and that control-referenced metrics are inflated by systematic variation [27], so our main references are the perturbed mean and additive composition.

3 Decoding a cell as a gene distribution

3.1 A cell as a distribution over a fixed gene panel

Let $x \in \mathbb{N}^G$ be the raw counts of one cell over a fixed panel of G genes, with library size $N = \sum_g x_g$, and let c be a text *state*: a cell type, or a perturbation together with an optional control cell x_0. We use G=2048 genes selected on training cells only (largest dispersion of log-normalized expression among genes detected in more than 1% of training cells; for Norman the training perturbation targets are always included). The panels contain no ribosomal-protein or mitochondrial genes.

Observation 1. *Let $N > 0$ and $p = x/N$. (i) $x \mapsto (N, p)$ is injective, so frequency and library size are lossless; if $x \sim \mathrm{Mult}(N, \pi)$, the log-likelihood of a model q is $N \sum_g p_g \log q_g + \text{const}$, maximized at $q = p$. (ii) A rank sentence (the ordering of genes by x) is constant on all count vectors with the same ordering, so it is not injective and keeps no magnitudes. (iii) Log1p or square-root vectors renormalized onto the simplex differ from p unless all nonzero counts are equal, so a model fit to them is not fit to the likelihood of x.*

(Proof in Appendix K.) We distinguish four objects. The decoder outputs $q(c) \in \Delta^{G-1}$, the *composition* of a cell in state c: $q_g(c)$ is the probability that a transcript of the cell, drawn at random from its panel UMIs, comes from gene g. It is not the probability that g is differentially expressed or that an intervention succeeds. Counts are drawn

as $x \mid N, q \sim \mathrm{Mult}(N, q)$. A decoder with a latent variable z outputs $q(c, z)$, and the population of cells in state c, $P(x \mid c)$, is then a mixture. A good fit of $q(c)$ to the mean composition does not by itself make $P(x \mid c)$ realistic, so we evaluate both.

3.2 A decision head on a language model

The state is written as a prompt and read by C2S-Scale-Gemma-2-2B [28, 29], a Gemma-2 model already trained on cell sentences, with rank-16 LoRA adapters on all linear layers [30]. The last-token hidden state h feeds a linear decision head with one logit per panel gene, and $q_\theta(c) = \mathrm{softmax}(Wh + \log b)$, where $\log b$ is an optional fixed log-prior: none, the smoothed training mean, or the smoothed control cell x_0, which makes the model a residual on the control cell. The head is initialized at zero, so the model starts as the prior and learns a correction. Training minimizes the cross-entropy $-\sum_g p_g \log q_{\theta,g}(c)$ between the frequency vector $p = x/N$ of a real training cell and the decoded q_θ (the log1p and square-root targets of Observation 1(iii) are worse on Norman for every metric; Appendix E). For perturbations without a control cell the prompt names the perturbed genes; with a control cell, each training perturbed cell is paired with a random training control cell, and the control cell enters through $\log b$ (Section 5.4).

What a categorical state can learn. When the state is a cell type, the objective averaged over the training cells of the type is minimized, by Gibbs' inequality, at their equal-weight mean frequency; the pooled score below is minimized at the UMI-weighted mean. A decoder conditioned on a label can therefore at best reproduce a type mean (a UMI-weighted loss did not help on validation; Appendix G).

3.3 Sampling cells and scoring them by likelihood

A cell is generated by drawing a library size from training cells and counts from $\mathrm{Mult}(s, q)$, or from a Dirichlet-multinomial with a concentration fit on training cells to represent overdispersion; every method uses the same samplers. For a held-out unit (a cell type or perturbation) with cells $x_1, \ldots, x_n$, we report $\ell(q) = -\sum_i \sum_g x_{ig} \log q_g / \sum_i N_i$, the multinomial negative log-likelihood (NLL) per UMI up to a model-independent constant; differences between models are exact differences in log-likelihood, and since the score is proper they equal differences in KL divergence to the true mean composition (Appendix K). Each perturbation receives the same $M = 50$ held-out control cells for every method. A model that conditions on a control cell gives one q_m per control cell; its *pooled* NLL uses $\bar{q} = \frac{1}{M}\sum_m q_m$ and its *cell* NLL is the mixture $-\sum_i \log \frac{1}{M}\sum_m \exp(\sum_g x_{ig} \log q_{mg}) / \sum_i N_i$, which rewards covering the variation between cells. A negative-binomial likelihood with gene-wise dispersions leaves the ordering of methods unchanged (Appendix C). Any method that outputs gene frequencies can be scored this way; methods with a native likelihood (scVI) are additionally scored with their own samples.

4 Data, baselines and evaluation

Conditional generation. Human PBMC from Hao et al. [31] (CELLxGENE Census; 161,159 cells of 8 donors and 29 cell types): donors P1–P5 train, P6 validation, P7–P8 test; units are cell types with at least 30 held-out cells. **Perturbation response.** The CRISPRa screen of Norman et al. in K562 [32, 33] (111,445 cells; 105 single and 131 paired perturbations). In our gene-level split, held-out single genes occur in no training perturbation (an earlier split leaked through training pairs; an audit found it and all its results were discarded). The 76 test perturbations form three strata: 21 unseen single genes, 29 pairs with one unseen gene, and 26 pairs of two genes seen alone in training.

Baselines. The global and cell-type means; the control and perturbed means; *partial additive* composition, in which an unseen gene contributes nothing; additive plus *target induction*, which multiplies each unseen target's frequency by one factor fit on training single perturbations (CRISPRa raises its target about 4.8-fold), the strongest cheap biology-aware baseline we know; ridge regression; scVI, scANVI and an scGen-style latent shift; GEARS; and autoregressive C2S on the same backbone and budget, whose sentences are mapped to frequencies by the C2S log-rank regression and mixed with the training mean (weight chosen on validation). All baselines are in frequency space and share the sampler.

Metrics and statistics. Besides NLL we report MMD and a real-versus-generated classifier AUC in a PCA space fit on training cells (lower is more realistic), and two biological measures on pseudobulk profiles $\log(1 + 10^4 \bar{q})$. Likelihood weights genes by their mass (TMSB4X alone carries about 16% of panel UMIs in K562), so for perturbations we report the *downstream* response: the top-20 differentially expressed genes of each perturbation (largest $|t|$ of a Welch

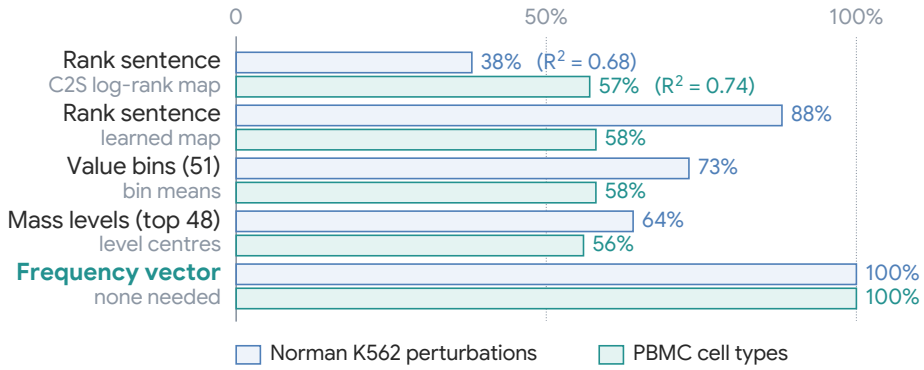


Figure 2: **The likelihood a rank sentence loses depends on how it is decoded.** Real held-out test cells were rewritten in each representation and decoded back to gene frequencies; bars give the share of the likelihood gain of the cells' own frequency vectors over the reference (perturbed mean for Norman, cell-type mean for PBMC) that survives, all estimated from the same cells in the same way. The C2S log-rank map keeps 38% on Norman although it reconstructs log expression with $R^2 = 0.68$, the criterion used to motivate rank sentences [2]; a map learned on training cells keeps 88%. Shares from decoding real cells, not model results; the learned map was added after the test results.

test between its real cells and held-out controls), *after removing the perturbed genes*, and on them r_{DE}, the Pearson correlation of predicted and real change over the control mean, and the fraction of genes changed in the right direction. The perturbed genes are scored separately as *target error*, the absolute error of their predicted log fold change. Units are cell types or perturbations; intervals are paired bootstraps over units. Decisions were made on validation; each test configuration was evaluated once per seed, under promotion rules written down before the test runs; analyses added after the test results are labeled as such (Appendix G).

5 Results

5.1 Q1: One forward pass beats generating a gene-rank sentence

Against Cell2Sentence on the same backbone and budget. We fine-tuned C2S-Scale-Gemma-2-2B autoregressively on rank sentences with the same LoRA rank, training cells, epochs, batch size and control cells as CellAsDistribution, and decoded its generated sentences with the C2S regression. The sentences are well formed (115 genes on average, 99.8% valid names), yet they add no likelihood beyond the perturbed mean (-0.001 nats per UMI, CI -0.006 to $+0.005$; the validation-chosen mixing weight is the largest on the grid, 0.9). CellAsDistribution is 0.103 nats per UMI better (CI 0.071 to 0.139) and recovers the downstream response far better (r_{DE} 0.77 against 0.56; Table 2). Decoding the same generated sentences with a rank-to-frequency map learned on training cells (described next) barely helps: CellAsDistribution remains 0.098 nats per UMI better (CI 0.069 to 0.131), with the same r_{DE} gap, so the shortfall lies in the generated sentences and not only in the map that decodes them (post hoc). Decoding costs one forward pass: alone on one A100 at batch size 25, C2S-Gemma needs 1.99 seconds per generated cell (530 new tokens on average), CellAsDistribution 0.004 seconds per cell with the control cell as a residual and 0.022 seconds when the control cell is also written as text, 90 to 540 times less.

Where a sentence loses likelihood. To separate the representation from the model, we rewrote the *real* held-out cells of each unit in a representation, decoded them back to frequencies, and asked how much of the likelihood gain of the cells' own frequency vectors survives (Figure 2). With the published log-rank map, the rank sentence reconstructs log expression well ($R^2 = 0.68$ on Norman, 0.74 on PBMC), yet keeps only 38% of the gain on Norman, against 73% for value bins and 64% for mass levels. A map learned on training cells, the mean frequency of each gene at each rank position, keeps 88% from the same sentences (90% when the library size is also given). Most of the loss is thus in the map from ranks to counts, not in the ranks: the ordering of a K562 cell's genes carries most of its composition, but a sentence has to be decoded, and the decoder decides how much survives; about a tenth is lost under every map we tried. On PBMC the representations do not separate (56–58% each), because cell types differ mostly in which genes

Δ to CellAsDistribution	all	single	1 unseen	both seen	r_{DE}
No LM (gene embeddings)	-0.005 [$-.017$, $+.009$]	-0.011	-0.003	-0.002	-0.012 [$-.042$, $+.016$]
Frozen LM + MLP head	+0.043 [+.025, +.060]	+0.008	+0.021	+0.096	-0.108 [$-.158$, $-.061$]
Additive	+0.009 [$-.010$, +.037]	+0.006	-0.010	+0.033	-0.004 [$-.040$, $+.033$]
Additive + target induction	+0.008 [$-.010$, +.036]	+0.003	-0.010	+0.033	-0.004 [$-.040$, $+.033$]

Table 1: Does the language model matter? Pooled NLL per UMI of each model minus that of CellAsDistribution without a control cell (same seed), Norman test split, seed 0: overall with [95% bootstrap over perturbations], and the mean difference by stratum (single $n = 21$, 1 unseen $n = 29$, both seen $n = 26$); positive values favor CellAsDistribution. r_{DE}: the same paired difference in downstream r_{DE} (perturbed genes excluded; negative favors CellAsDistribution). Every model uses the same 20,000 training cells, frequency target, training-mean log-prior, softmax over the panel and scoring. Overall NLL difference by seed (no LM / frozen LM): seed 0: -0.005 / +0.043; seed 1: -0.007 / +0.044; seed 2: -0.003 / +0.047. Pre-registered (Appendix G).

are expressed. A generator that outputs the composition needs no such map, and its likelihood is defined by the model itself.

5.2 Q2: The gain comes from the output, not the language model

We compared CellAsDistribution without a control cell against two controls, specified before they were run (Table 1), with the same training cells, target, log-prior, softmax output and scoring: the same language model *frozen*, with only an MLP head trained on its last-token state; and a decoder with *no language model*, in which a perturbation is the sum of learned embeddings of its genes (an unseen gene gets a zero embedding) followed by an MLP. On test, the decoder without a language model ties CellAsDistribution in every stratum and for every seed (overall -0.005 nats per UMI, CI -0.017 to $+0.009$; seeds 1 and 2: -0.007 and -0.003) and on the downstream response (r_{DE} -0.012, CI -0.042 to $+0.016$), with 1.1 million parameters trained in 10 seconds. The frozen language model is worse ($+0.043$, CI $+0.025$ to $+0.060$; r_{DE} -0.108), most on pairs of seen genes ($+0.096$). (On validation, with the Qwen backbone, the no-language-model decoder was 0.013 nats worse, CI 0.000 to 0.028.) By the rule we fixed before these runs, evaluated on test, the likelihood gain over C2S and the perturbed mean comes from decoding the distribution directly, not from knowledge in the language model. Fine-tuning is what makes the language model as good a decoder of the condition as a purpose-built encoder; what the language model adds is the interface, one prompt format for any state, which is what lets the same model generate cell types it has never seen (Section 5.5). The prompts tell the same story. Replacing gene symbols by opaque identifiers in the prompt worsens the downstream response even on pairs of seen genes (r_{DE} 0.87 against 0.89, overall 0.69 against 0.77), but since an identity-only embedding without any text matches the model that reads real names, the names act as identifiers, and opaque tokens that share sub-words (GENE001, GENE002, ...) blur them; on unseen single genes the names bring nothing measurable ($+0.007$ nats, CI -0.022 to $+0.030$), nor do NCBI gene summaries (validation). The backbone matters modestly: across three seeds the C2S-initialized Gemma is 0.011 to 0.012 nats better than the general-purpose Qwen3.5-0.8B on both tasks, more than the seed spread (0.001 to 0.003), without changing any ranking; LoRA ranks 8 to 64 give the same NLL.

5.3 Q3a: Downstream response matches additive; target induction is shrunk

Likelihood and realism (Table 2). Without a control cell, CellAsDistribution is 0.104 nats per UMI better than the perturbed mean (CI 0.071 to 0.139) and ties the additive baseline (-0.009, CI -0.037 to $+0.010$). Its generated cells are closer to real ones than additive cells (MMD 0.039 against 0.052; AUC 0.81 against 0.86), but not than those of the scGen-style shift, which moves each control cell individually. GEARS with default settings is 0.47 nats above the perturbed mean after its regression output is mapped to frequencies (Appendix J); this measures that conversion as much as the model.

Downstream response, target gene set aside (Table 3; post hoc). CRISPRa raises its own target, so a model can look right on the target alone. With the perturbed genes removed, CellAsDistribution reproduces the direction of 89% of the top-20 downstream genes and ties additive on r_{DE} overall (0.77 against 0.77; $+0.004$, CI -0.033 to $+0.040$) and in every stratum, and is better than scGen-style (0.72) and C2S (0.56). On the target genes themselves it is *worse* than additive on pairs (target error 0.51 against 0.27 with one unseen gene, 0.58 against 0.39 with both seen).

Model	ΔNLL to additive	cell NLL	MMD	AUC	r_{DE}	sign
Perturbed mean	+0.094 [+.058, +.136]	5.289	0.073	0.903	0.58	0.81
Additive (reference)	0	5.195	0.052	0.855	0.77	0.90
Additive + target induction	−0.001 [−.003, +.000]	5.194	0.051	0.857	0.77	0.90
scGen-style	+0.006 [−.014, +.022]	5.105	0.029	0.769	0.72	0.85
C2S, autoregressive	+0.094 [+.059, +.133]	5.279	0.075	0.906	0.56	0.81
Additive, control residual	+0.005 [−.001, +.011]	5.134	0.038	0.815	0.76	0.89
CellAsDistribution, no control cell	−0.009 [−.037, +.010]	5.186	0.039	0.812	0.77	0.89
CellAsDistribution, control residual	+0.002 [−.025, +.022]	5.129	0.030	0.785	0.72	0.86
CellAsDistribution, NB head	+0.006 [−.021, +.028]	5.201	0.036	0.797	0.77	0.88
CellAsDistribution, low-rank head	+0.053 [+.024, +.084]	5.096	0.040	0.791	0.73	0.87

Table 2: Perturbation response, Norman K562 test split (76 held-out perturbations, matched control cells, seed 0, C2S-Scale-Gemma-2-2B for every language model). ΔNLL: pooled NLL per UMI minus the additive baseline's [95% bootstrap over perturbations; lower is better; the additive baseline's NLL is 5.195]. cell NLL: mixture over the $M = 50$ control cells (or the low-rank head's draws). MMD and real-versus-generated AUC (lower is better) on Dirichlet-multinomial samples. Downstream biology, with the perturbed genes removed (added after the test results; post hoc): r_{DE}, Pearson correlation of predicted and real pseudobulk change over the held-out control mean on the perturbation's top-20 differentially expressed genes; sign, the fraction of those genes whose change has the right sign.

	ΔNLL to additive			downstream r_{DE}			target error	
Model	single	1 unseen	2 seen	single	1 unseen	2 seen	1 unseen	2 seen
Additive	0	0	0	0.51	0.83	0.90	0.27	0.39
Additive + target induction	−0.003	+0.000	−0.000	0.51	0.83	0.90	0.24	0.39
scGen-style	−0.002	+0.037	−0.021	0.44	0.79	0.87	0.74	0.65
CellAsDistribution, no control cell	−0.006	+0.010	−0.033	0.54	0.83	0.89	0.51	0.58
CellAsDistribution, opaque gene ids	+0.001	+0.029	−0.019	0.42	0.73	0.87	0.72	0.76

Table 3: Norman test split by stratum: unseen single genes ($n = 21$), pairs with one unseen gene ($n = 29$) and pairs of two genes seen alone in training ($n = 26$). ΔNLL: mean paired difference to the additive baseline (intervals in the text). Downstream r_{DE} as in Table 2, perturbed genes excluded. Target error: mean absolute error of the predicted log fold change of the perturbed gene(s) themselves over the held-out control mean, for targets in the panel. Added after the test results (post hoc).

The error has a direction: the real induction of a target is about 1.7 log units, additive composition copies it from the training single perturbation almost without bias ($+0.04$ to $+0.09$), and CellAsDistribution under-predicts it by about 0.5, shrinking a large gene-specific change toward the mean composition it learns a correction to (only 4 of the 21 unseen single targets are in the panel, so this concerns pairs). Adding target induction to the additive baseline lowers its target error further but changes its NLL by at most 0.003 nats, because a target gene carries little transcript mass; likelihood and target accuracy are separate properties and we report both.

Genes never perturbed in training. On the 21 genes never perturbed in training no method in our comparison does well: CellAsDistribution ties the perturbed mean (-0.006, CI -0.025 to $+0.018$; r_{DE} 0.54 against 0.51), while a split-half oracle lies 0.108 nats below it (Appendix A). Figure 3 shows the median unit of each extreme stratum. For the median unseen gene, MAML2, its own induction is the largest change among its top-20 DE genes, and neither CellAsDistribution nor any baseline predicts it: a decoder that has never seen the gene perturbed cannot infer this from its name. For the median pair of seen genes, DUSP9+SNAI1, CellAsDistribution reproduces the downstream program in direction and rank (Pearson 0.91 on the top-20 DE genes, against 0.95 for additive), while compressing the largest changes, the same shrinkage as for target genes.

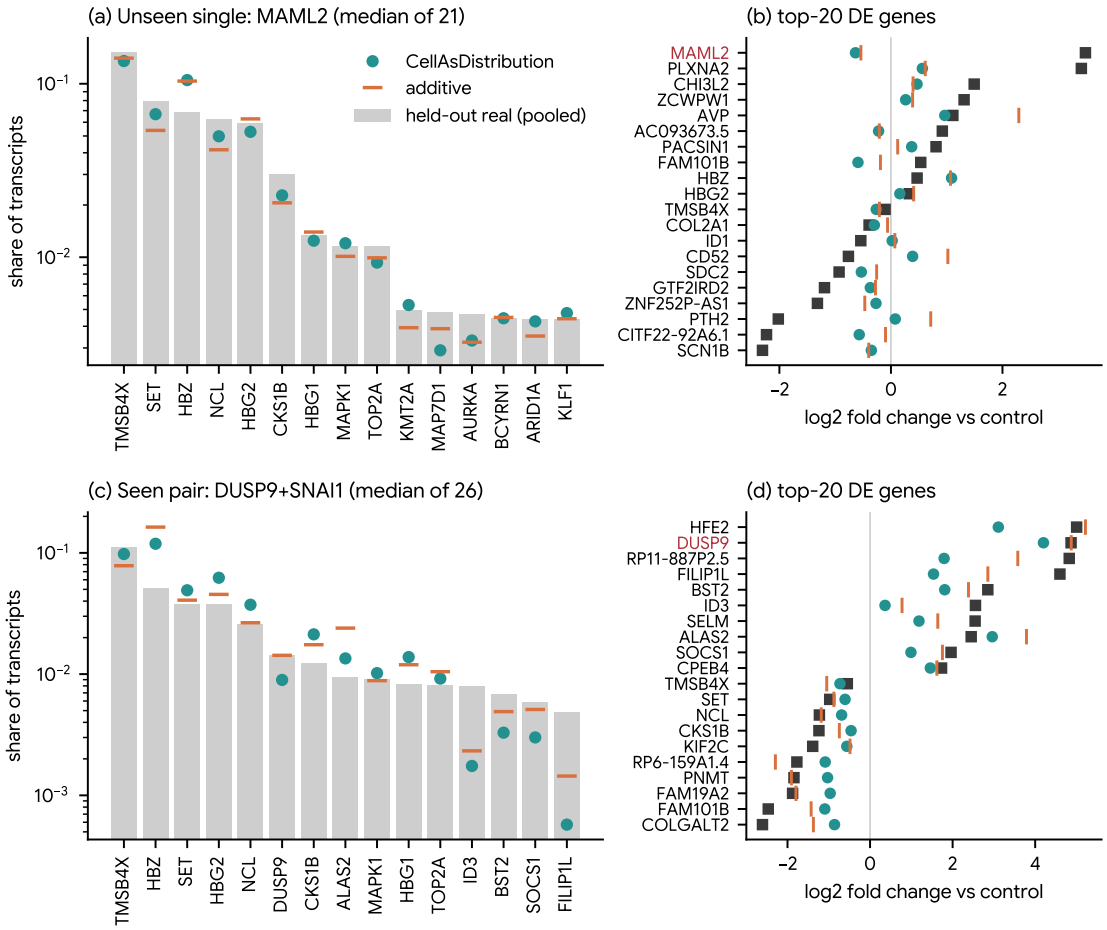


Figure 3: **What the decoder gets right and wrong, on two median test perturbations** (the unit with the median paired NLL difference to additive in its stratum, so not hand-picked; Norman test split, Gemma, no control cell, seed 0). (a, c) Share of transcripts of the 15 most abundant genes in the held-out real cells (grey), predicted by CellAsDistribution (teal) and by additive composition (orange). (b, d) Log fold change over the held-out control mean of the perturbation's top-20 DE genes (Welch t on real cells), perturbed genes in red. (a, b) MAML2, never perturbed in training: no model predicts its self-induction; CellAsDistribution and additive (here the perturbed mean) are both near the mean composition. (c, d) DUSP9+SNAI1, both genes seen alone in training: CellAsDistribution recovers the direction and order of the downstream changes but compresses the largest ones.

5.4 Q3b: Cell-to-cell variation enters through the control cell

A single q per perturbation leaves all cell-to-cell variation to the sampler. With the control cell as the log-prior ($b = \mathrm{smooth}(x_0)$, not written in the prompt), the cell NLL improves from 5.186 to 5.129 and the cells become more realistic (MMD 0.030, AUC 0.79), approaching scGen-style on MMD. Writing the control cell into the prompt instead loses this gain ($+0.019$ nats per UMI in cell NLL, CI $+0.012$ to $+0.027$). A version of the residual without a language model, in which the additive shift replaces the learned correction, reaches the same likelihood (5.134) but less realistic cells (MMD $+0.008$, AUC $+0.030$ relative to CellAsDistribution). The residual has a cost: its downstream r_{DE} falls to 0.72, below additive (-0.046, CI -0.092 to -0.008), because each control cell's own deviations enter the prediction. Two heads keep the single pass and the normalization of q while modeling variation the flat softmax lacks (Appendix H): a low-rank logistic-normal head, $q(c, z)$ along 16 learned gene programs, lowers the cell NLL to 5.096 without any control cell at the cost of the pooled NLL ($+0.062$); a negative-binomial head with per-gene dispersion improves the change correlation over all genes (r_Δ $+0.020$) at a small cost in NLL. Gene–gene correlations

Model	ΔNLL to type mean	cell NLL	MMD	AUC	type acc.
Global mean	+1.125 [+.939, +1.326]	6.732	0.833	1.000	0.04
Cell-type mean (reference)	0	5.606	0.182	0.985	0.88
scVI	+0.005 [−.007, +.016]	5.611	0.166	0.975	0.82
scVI, own NB sampling	−0.001 [−.011, +.010]	5.489	0.128	0.924	0.70
scANVI, own NB sampling	+0.003 [−.009, +.017]	5.486	0.130	0.930	0.67
CellAsDistribution	+0.017 [+.001, +.036]	5.623	0.163	0.970	0.82
CellAsDistribution, low-rank head	+0.041 [+.018, +.067]	5.552	0.165	0.970	0.77

Table 4: Conditional generation, human PBMC test split (unseen donors; 24 cell types; seed 0; Gemma backbone). ΔNLL: paired difference to the cell-type mean [95% bootstrap over cell types]. *Own NB sampling*: cells drawn from scVI/scANVI's own negative binomial from latent states of training cells, with cell NLL as the mixture over these draws; other rows use the shared samplers (generation metrics on Dirichlet-multinomial samples).

Unseen cell type	ΔNLL to global mean	ΔNLL to name overlap	type acc.
Nearest Cell Ontology type	−0.209 [−.612, +.328]	+0.440 [+.133, +.860]	0.02
Cell Ontology parents	−0.515 [−.737, −.285]	+0.134 [+.001, +.299]	0.05
Name-overlap type (string match)	−0.649 [−.877, −.388]	0	0.02
CellAsDistribution, opaque type id	+0.106 [−.154, +.377]	+0.754 [+.470, +1.041]	0.01
CellAsDistribution, type name	−0.707 [−.875, −.542]	−0.059 [−.216, +.097]	0.15
CellAsDistribution, name + definition	−0.707 [−.862, −.545]	−0.058 [−.219, +.101]	0.14

Table 5: Zero-shot cell types, PBMC test split: 19 types, each held out of training in one of five leave-types-out folds (Qwen3.5-0.8B backbone, seed 0). Paired differences [95% bootstrap over cell types]. Baselines use training types only.

of generated cells on 100 variable genes are close to those of real cells for every method at this sequencing depth (Appendix I).

5.5 Q3c: Cell types: the type mean for known types, names for unseen ones

Known types (Table 4). As the remark of Section 3.2 predicts, no model improves on the cell-type mean in likelihood on unseen donors: CellAsDistribution is slightly worse ($+0.017$, CI $+0.001$ to $+0.036$), scVI and scANVI tie it, and on validation CellAsDistribution gives the same NLL to four decimals for every LoRA rank and both backbones. Sample realism is decided by the source of cell-to-cell variation: scVI sampling from its own negative binomial reaches the lowest MMD and AUC (0.92), every frequency decoder relies on the shared sampler, and no generator is near indistinguishable from real cells of unseen donors (AUC ≥ 0.92).

Unseen types (Table 5). A model indexed by cell type cannot generate a type it has never seen; a decoder that reads the type's name can. In five leave-types-out folds over 19 types, CellAsDistribution generates each held-out type from its name and Cell Ontology definition. It is 0.707 nats per UMI better than the global mean (CI 0.545 to 0.862) and better than the nearest Cell Ontology type and the pooled Cell Ontology parents; it ties a string-matching baseline that copies the training type whose name shares the most words (-0.058, CI -0.219 to $+0.101$) but recovers the requested type far more often (0.14 against 0.02). The definition adds nothing beyond the name, and opaque identifiers remove the gain: CellAsDistribution generalizes by composing the words of type names, and we find no evidence that it uses knowledge beyond them.

6 Discussion and limitations

Letting a decision language model answer with a gene distribution makes a language model a single-cell generator whose output is a cell's composition rather than a sentence about it. The interface has concrete consequences: generation costs one forward pass, the model's likelihood is defined without a reconstruction map, and every method can

be compared on one proper score. The comparison with rank sentences shows why the last point matters: the same sentences keep 38% or 88% of the achievable likelihood depending on the map chosen to decode them, so a sentence model's quality cannot be separated from a post hoc choice, whereas a distribution is scored as produced. Our controls are equally clear about what the language model does not do: a decoder of learned gene embeddings with the same output matches it on perturbations of known genes, so the decision-native output, not language-model knowledge, carries the gain; the language model makes the interface general, from perturbation names to cell-type names and definitions, and that generality is what enables generation of unseen cell types.

A better gene distribution is not yet better biology, and our biological results draw a clear boundary. Where training contains the perturbed genes, the decoder matches additive composition on likelihood and on the downstream program, and generates more realistic cells; it is worse on the size of the target's own induction; and it does not transfer to genes it has never seen perturbed, where neither its names nor gene descriptions help. A categorical cell type caps any decoder at the type mean, while names let it compose unseen types. The output q is a composition, not a probability of differential expression or of an intervention's success; a decision head trained on such events, with calibration measured on held-out perturbations, is the natural next step for the decision-native interface, together with richer states (donor, cell and pathway context) and heads that model dispersion and covariance jointly.

Limitations. One dataset per task, a 2048-gene panel and CRISPRa only; held-out control cells are not paired with perturbed cells, so cell NLL rewards covering cell states rather than predicting a counterfactual; GEARS was run with default settings and CPA and the transport and flow models were not compared, because they do not output a distribution over a fixed gene list; zero-shot types were evaluated with the Qwen backbone; the downstream, target, co-expression and learned-rank-map analyses were added after the test results (Appendix G).

Code availability. Code, held-out unit lists, run scripts and table-generation scripts are available with a README at https://github.com/kagtgi/CellAsDistribution.

AI use statement. Code, experiments, tables, figures and a first draft of this text were produced with assistance from Claude (Anthropic); the authors reviewed all content and are responsible for it.

References

[1] Charlotte Bunne, Yusuf Roohani, Yanay Rosen, et al. How to build the virtual cell with artificial intelligence: Priorities and opportunities. *Cell*, 187:7045–7063, 2024. doi: 10.1016/j.cell.2024.11.015.

[2] Daniel Levine, Syed A. Rizvi, Sacha Lévy, et al. Cell2Sentence: Teaching large language models the language of biology. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 27299–27325, 2024. URL https://proceedings.mlr.press/v235/levine24a.html.

[3] Syed Asad Rizvi, Daniel Levine, Aakash Patel, et al. Scaling large language models for next-generation single-cell analysis. bioRxiv, 2025.

[4] Unsloth. Train your own decision model with Unsloth. Documentation, 2026. URL https://unsloth.ai/docs/basics/train-your-own-decision-model-with-unsloth.

[5] Diogo Almeida. Introducing system one models & Jev. TypeSafe AI blog, 15 September, 2026. URL https://typesafe.ai/blog/introducing-system-one-models-and-jev.

[6] Constantin Ahlmann-Eltze, Wolfgang Huber, and Simon Anders. Deep-learning-based gene perturbation effect prediction does not yet outperform simple linear baselines. *Nature Methods*, 22:1657–1661, 2025. doi: 10.1038/s41592-025-02772-6.

[7] Zhiting Wei, Yiheng Wang, Yicheng Gao, et al. Benchmarking algorithms for generalizable single-cell perturbation response prediction. *Nature Methods*, 23:451–464, 2026. doi: 10.1038/s41592-025-02980-0.

[8] Tobias Deußer, Lorenz Sparrenberg, and Rafet Sifa. Evaluating and benchmarking the system one model Jev. arXiv:2609.37647, 2026.

[9] Haotian Cui et al. scGPT: toward building a foundation model for single-cell multi-omics using generative AI. *Nature Methods*, 21:1470–1480, 2024. doi: 10.1038/s41592-024-02201-0.

[10] Christina V. Theodoris et al. Transfer learning enables predictions in network biology. *Nature*, 618:616–624, 2023. doi: 10.1038/s41586-023-06139-9.

[11] Yanay Rosen, Yusuf Roohani, Ayush Agrawal, et al. Universal cell embedding provides a foundation model for cell biology. *Nature*, 656(8126):183–191, 2026. doi: 10.1038/s41586-026-10689-z.

[12] Michael M. Danziger, Bharath Dandala, Viatcheslav Gurev, et al. BMFM-RNA: whole-cell expression decoding improves transcriptomic foundation models. arXiv:2506.14861, 2025.

[13] Romain Lopez, Jeffrey Regier, Michael B. Cole, Michael I. Jordan, and Nir Yosef. Deep generative modeling for single-cell transcriptomics. *Nature Methods*, 15:1053–1058, 2018. doi: 10.1038/s41592-018-0229-2.

[14] Chenling Xu, Romain Lopez, Edouard Mehlman, et al. Probabilistic harmonization and annotation of single-cell transcriptomics data with deep generative models. *Molecular Systems Biology*, 17(1):e9620, 2021. doi: 10.15252/msb.20209620.

[15] Erpai Luo, Minsheng Hao, Lei Wei, et al. scDiffusion: conditional generation of high-quality single-cell data using diffusion model. *Bioinformatics*, 40(9):btae518, 2024. doi: 10.1093/bioinformatics/btae518.

[16] Riccardo Bellazzi, Andrea Codegoni, Stefano Gualandi, Giovanna Nicora, and Eleonora Vercesi. The Gene Mover's Distance: Single-cell similarity via optimal transport. arXiv:2102.01218, 2021.

[17] Mohammad Lotfollahi, F. Alexander Wolf, and Fabian J. Theis. scGen predicts single-cell perturbation responses. *Nature Methods*, 16:715–721, 2019. doi: 10.1038/s41592-019-0494-8.

[18] Mohammad Lotfollahi, Anna Klimovskaia Susmelj, Carlo De Donno, et al. Predicting cellular responses to complex perturbations in high-throughput screens. *Molecular Systems Biology*, 19(6):e11517, 2023. doi: 10.15252/msb.202211517.

[19] Yusuf Roohani, Kexin Huang, and Jure Leskovec. Predicting transcriptional outcomes of novel multigene perturbations with GEARS. *Nature Biotechnology*, 42:927–935, 2024. doi: 10.1038/s41587-023-01905-6.

[20] Abhinav K. Adduri, Dhruv Gautam, Beatrice Bevilacqua, et al. Predicting cellular responses to perturbation across diverse contexts with State. *Cell*, 189(19):5914–5931.e20, 2026. doi: 10.1016/j.cell.2026.07.052.

[21] Handong Wang, Jiaxin Qi, Haochen Feng, and Baisheng Lai. PopPert: Population-level joint-distribution modeling for single-cell perturbation prediction. arXiv:2609.01357, 2026.

[22] Chenglei Yu, Chuanrui Wang, Bangyan Liao, and Tailin Wu. scDFM: Distributional flow matching model for robust single-cell perturbation prediction. In *International Conference on Learning Representations*, 2026. arXiv:2602.07103.

[23] Shuizhou Chen, Lang Yu, Xueqin Lin, et al. SCALE: Scalable conditional atlas-level endpoint transport for virtual cell perturbation prediction. arXiv:2603.17380, 2026.

[24] Bowen Han, Lingbei Meng, Shihuan Luo, et al. TempoBridge: Source-conditioned flow matching with optimal transport couplings for single-cell population transitions. arXiv:2610.04945, 2026.

[25] Changxi Chi, Yufei Huang, Jun Xia, et al. Departures: Distributional transport for single-cell perturbation prediction with neural Schrödinger bridges. arXiv:2511.13124, 2025.

[26] Danning Jiang, Zhiwen Yan, Qirun Wang, et al. OCOO-T: A simple and scalable virtual cell model for transcriptional perturbation response prediction. arXiv:2606.12838, 2026.

[27] Ramon Viñas Torné, Maciej Wiatrak, Zoe Piran, et al. Systema: a framework for evaluating genetic perturbation response prediction beyond systematic variation. *Nature Biotechnology*, 44(6):1050–1059, 2026. doi: 10.1038/s41587-025-02777-8.

[28] van Dijk Lab. C2S-Scale-Gemma-2-2B (model card). Hugging Face, vandijklab/C2S-Scale-Gemma-2-2B, 2025. URL https://huggingface.co/vandijklab/C2S-Scale-Gemma-2-2B.

[29] Gemma Team. Gemma 2: Improving open language models at a practical size. arXiv:2408.00118, 2024.

[30] Edward J. Hu, Yelong Shen, Phillip Wallis, et al. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. arXiv:2106.09685.

[31] Yuhan Hao et al. Integrated analysis of multimodal single-cell data. *Cell*, 184:3573–3587.e29, 2021. doi: 10.1016/j.cell.2021.04.048.

[32] Thomas M. Norman, Max A. Horlbeck, Joseph M. Replogle, et al. Exploring genetic interaction manifolds constructed from rich single-cell phenotypes. *Science*, 365:786–793, 2019. doi: 10.1126/science.aax4438.

[33] Stefan Peidli, Tessa D. Green, Ciyue Shen, et al. scPerturb: harmonized single-cell perturbation data. *Nature Methods*, 21:531–540, 2024. doi: 10.1038/s41592-023-02144-y.

Supplementary material

CellAsDistribution: Decision-Native Language Modeling of Single-Cell Gene Composition

A Gene-level five-fold analysis (Norman)

This analysis was run with the Qwen3.5-0.8B backbone (CAD-P, the flat decoder of the population-level analysis) before the Gemma backbone was added. Pooled over five gene-level folds, the flat decoder ties the partial-additive baseline on pairs of seen genes (-0.017, CI -0.060 to $+0.021$, $n = 28$) and on pairs with one unseen gene ($+0.004$, CI -0.014 to $+0.022$, $n = 51$), and it beats ridge. The split-half oracle lies 0.108 nats below the perturbed mean on unseen single genes, so headroom exists; descriptions and shuffled names do not help, and shuffled names are harmful ($+0.045$). A one-parameter target-induction baseline, which raises the unseen target gene's frequency, is better than every learned decoder on the 75 genes whose target is in the panel (-0.013, CI -0.031 to -0.004). On the test split, CAD-P (Qwen backbone) is 0.097 better than the perturbed mean and ties additive (-0.003, CI -0.031 to $+0.019$).

B Training details

Table 7 lists the training settings of the decision language models, and the paragraphs below it describe the scVI-based baselines.

scVI and scANVI. Both are trained on training cells of the panel with a negative-binomial likelihood, 30 latent dimensions and 60 epochs (scANVI: 15 further epochs with cell-type labels). The frequency vector of a cell type is the mean decoder output over up to 3000 training cells of that type.

scGen-style baseline. An scVI model is trained on all training cells. The latent shift of a single gene is the mean latent state of its perturbed cells minus that of the control cells, the shift of a pair is the sum of the shifts of its known genes, and a perturbation with no known gene uses the mean shift of all training perturbations. Each held-out control cell is encoded, shifted and decoded with library size one.

C Negative-binomial evaluation

For a held-out cell i of a unit with library size s_i and model frequency q, the count of gene g is scored as $\mathrm{NB}(x_{ig} \mid \mu = s_i q_g, \phi_g)$ with variance $\mu + \phi_g \mu^2$. The dispersions ϕ_g come from a method-of-moments fit on library-normalized training counts with the within-unit variance pooled over units (median $\phi = 26.5$ for PBMC and 9.0 for Norman, clipped to $[10^{-3}, 10^2]$). The score is the negative log-likelihood per UMI, which is on a different scale from the multinomial score because it includes the zeros, so only differences are compared (Table 8). It uses the pooled frequency of a unit, so it does not reward conditioning on a control cell.

On PBMC the negative-binomial score separates the decoders slightly more than the multinomial score does: CAD is reliably, but marginally (0.004 nats), worse than the cell-type mean. On Norman the ordering of the methods is the same as under the multinomial score.

D All worked examples: prompts and decoded distributions

Figures 5 and 6 show, for every use case, the exact prompt that the decision language model reads and the distribution it returns, next to the held-out ground truth. The examples are not hand-picked. In each case, the shown unit is the held-out cell type or perturbation whose paired NLL difference between CellAsDistribution and the case's reference is the median over the units of that case, so half of the units look better and half look worse. The cases are:

- conditional generation on unseen PBMC donors, against the cell-type mean;
- perturbation response without a control cell, one example per stratum of the gene-level split (unseen single gene, pair with one unseen gene, pair of two seen genes), against the additive baseline;

Paired NLL/UMI difference [95% CI]	unseen singles ($n = 84$)	pairs (seen genes) ($n = 28$)	pairs (one unseen gene) ($n = 51$)
to the perturbed mean			
additive (partial)	+0.000 [+0.000, +0.000]	-0.127 [-0.186, -0.077]	-0.076 [-0.111, -0.044]
ridge	**-0.002 [-0.010, +0.006]**	-0.033 [-0.053, -0.013]	-0.015 [-0.025, -0.006]
CAD-P, real gene names	+0.007 [-0.010, +0.023]	-0.144 [-0.197, -0.095]	-0.072 [-0.101, -0.046]
CAD-P, opaque ids	+0.015 [+0.002, +0.031]	-0.147 [-0.208, -0.099]	-0.066 [-0.090, -0.042]
CAD-P, shuffled names	+0.045 [+0.027, +0.063]	-0.155 [-0.208, -0.108]	-0.053 [-0.079, -0.027]
CAD-P + own description	+0.007 [-0.008, +0.022]	**-0.157 [-0.211, -0.112]**	**-0.084 [-0.112, -0.057]**
CAD-P + other gene's description	+0.018 [+0.003, +0.033]	-0.153 [-0.197, -0.114]	-0.072 [-0.102, -0.045]
split-half oracle (reference)	-0.108 [-0.147, -0.075]	-0.155 [-0.215, -0.105]	-0.151 [-0.188, -0.118]
between decoders			
real − opaque ids (primary)	-0.008 [-0.029, +0.011]	+0.003 [-0.019, +0.030]	-0.006 [-0.025, +0.013]
real − shuffled names	-0.038 [-0.061, -0.016]	+0.011 [+0.000, +0.025]	-0.019 [-0.046, +0.004]
description − other description	-0.011 [-0.022, -0.001]	-0.003 [-0.024, +0.012]	-0.012 [-0.022, -0.000]
description − real names	+0.000 [-0.010, +0.011]	-0.013 [-0.026, -0.002]	-0.012 [-0.024, -0.000]
CAD-P − additive	+0.007 [-0.010, +0.023]	-0.017 [-0.060, +0.021]	+0.004 [-0.014, +0.022]
CAD-P − ridge	+0.009 [-0.008, +0.025]	-0.111 [-0.167, -0.062]	-0.057 [-0.084, -0.033]

Table 6: Gene-level 5-fold analysis on Norman (CAD-P, Qwen3.5-0.8B backbone): pooled paired NLL differences (nats per UMI; negative favors the first term). Unit: perturbation.

Setting	Value
Optimizer, schedule	AdamW (weight decay 0), one-cycle schedule with 5% warm-up, gradient clipping at 1.0
Learning rates	2×10^{-4} (LoRA adapters), 10^{-3} (decision head, zero-initialized)
Batch size, budget	16; one epoch over 20,000 training cells drawn at random (1,250 steps)
LoRA	rank 16, $\alpha = 16$, dropout 0, all linear layers; rank 8 and 64 for the sensitivity run
Precision, attention	bf16; eager attention and gradient checkpointing for Gemma-2
Output vocabulary	the 2048 panel genes as classes; no new tokens, gene names are tokenized as plain text
Control-cell text	48 most abundant genes with 14 log-mass levels; 200 gene names for the rank variant
Seeds	test: Gemma and Qwen 0, 1, 2; validation: Gemma 3407 (default), 1, 2; Qwen 1, 2; variants of Appendix G: Qwen seed 0 (the seed changes the sampled training cells and the sampler)
Negative-binomial head	inverse dispersion $\log \theta_g(c) = \beta_g + v_g^\top h$, initialized at $\theta = 1$ and clipped to $[e^{-6}, e^{8}]$; loss: negative-binomial NLL of the raw panel counts with mean $N_i q_g$, per UMI
Low-rank head	$k = 16$ programs; amplitude $a(h) = \mathrm{softplus}(\cdot)$ initialized at 1; encoder of the target cell: $\log(1 + 10^4 x/N) \to 256$ (GELU) $\to 2k$; one reparameterized sample per cell; KL weight raised linearly from 0 to 1 over the first 30% of steps
C2S-Gemma baseline	C2S-Scale-Gemma-2-2B, LoRA $r = 16$ on all linear layers, lr 2×10^{-4}, one-cycle schedule, 20,000 cells, one epoch, batch 16 (micro-batches of 2); control and target as top-200 rank sentences; loss on the response only; sampling with top-k 50, top-p 0.95, up to 1024 new tokens
Evaluation	$M = 50$ held-out control cells per perturbation, the same for every method (earlier runs: drawn separately per method); 300 generated cells per unit
Hardware, time	one A100 40 GB; run alone, Qwen takes 6 to 8.5 min and Gemma 9.5 to 24 min per run (90 to 100 min with the 200-gene rank text; wall-clock was longer when jobs shared the GPU); Gemma peak memory 8.7 GB

Table 7: Training settings of the decision language models.

- perturbation response with a control cell, for a pair of two seen genes, once with the control cell both written as mass-level text in the prompt and used as the residual prior (the prompt shows a real serialized control cell), and once with the control cell used only as the residual prior;
- zero-shot cell types (fold 0, so the median of its 4 held-out types), against the mean of the nearest training type in the Cell Ontology.

The ground truth is the pooled frequency of all held-out real cells of the unit. For a control-cell model the plotted distribution is the mean over the $M = 50$ matched control cells, and the prompt shown is the one built from the first of them. For perturbations, the right panel compares the log fold change over the held-out control mean on the unit's top-20 differentially expressed genes (largest $|t|$ of a Welch test between its real cells and the held-out control cells, computed on real cells only). The selected units and the numbers quoted for them are:

(a) *PBMC, conditional generation*: naive thymus-derived CD4-positive, alpha-beta T cell, the median of 24 units (NLL difference to the cell-type mean: $+0.011$ nats per UMI).

(b) *K562, no control cell, unseen single gene*: MAML2, the median of 21 units (NLL difference to the additive baseline: -0.015 nats per UMI); Pearson correlation with the real log fold change on its top-20 DE genes: 0.33

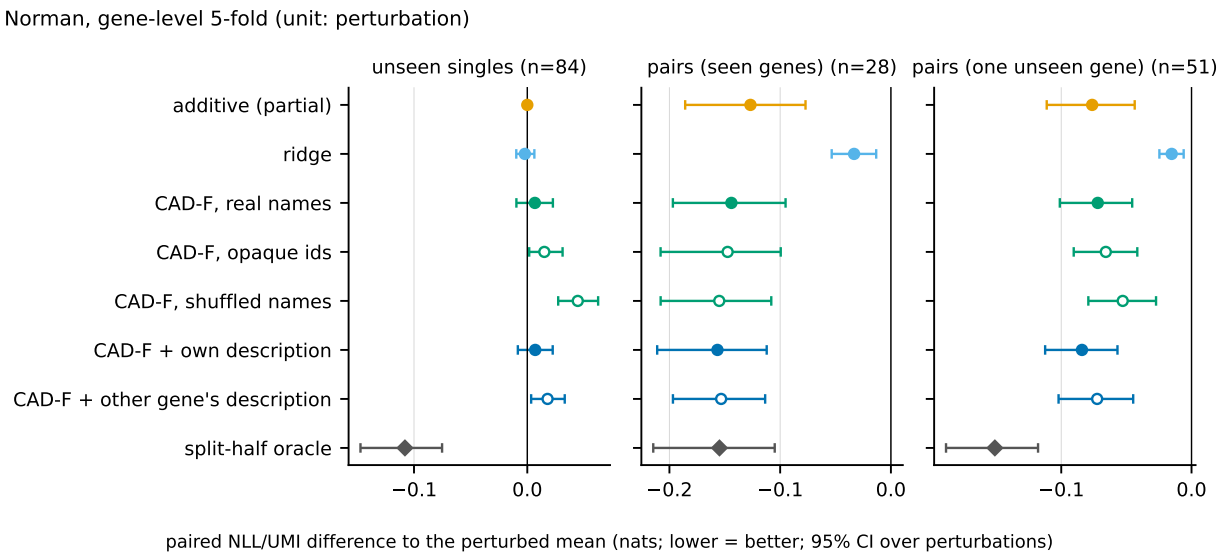


Figure 4: Gene-level 5-fold on Norman, population level: paired held-out NLL difference to the perturbed mean (bars: 95% bootstrap over perturbations; hollow: identity or knowledge controls; diamond: split-half oracle).

Task	Model	ΔNB [95% CI]	Δmultinomial
PBMC (vs. cell-type mean)	global mean	+0.302 [+0.252, +0.352]	+1.125
	scVI	+0.004 [+0.001, +0.007]	+0.005
	scANVI	+0.002 [+0.000, +0.005]	+0.004
	CAD (Gemma)	+0.004 [+0.002, +0.006]	+0.017
	CAD (Qwen)	+0.005 [+0.002, +0.009]	+0.030
Norman (vs. perturbed mean)	additive	−0.014 [−0.023, −0.004]	−0.094
	ridge	−0.008 [−0.010, −0.006]	−0.024
	scGen-style	−0.014 [−0.021, −0.008]	−0.085
	CAD (Gemma, no control)	−0.021 [−0.027, −0.015]	−0.104
	CAD (Gemma, control)	−0.017 [−0.023, −0.012]	−0.091
	CAD (Qwen, no control)	−0.019 [−0.025, −0.013]	−0.092
	CAD (Qwen, control)	−0.015 [−0.020, −0.010]	−0.078

Table 8: Negative-binomial and multinomial NLL per UMI relative to the reference (test split, seed 0; earlier runs, with control cells not matched across methods; negative favors the model; paired bootstrap over cell types or perturbations).

for CAD and 0.19 for the reference.

(c) *K562, no control cell, pair with one unseen gene*: FOXF1+FOXL2, the median of 29 units (NLL difference to the additive baseline: +0.011 nats per UMI); Pearson correlation with the real log fold change on its top-20 DE genes: 0.92 for CAD and 0.96 for the reference.

(d) *K562, no control cell, pair of two seen genes*: DUSP9+SNAI1, the median of 26 units (NLL difference to the additive baseline: −0.012 nats per UMI); Pearson correlation with the real log fold change on its top-20 DE genes: 0.91 for CAD and 0.95 for the reference.

(e) *K562, control cell as text and residual, pair of two seen genes*: ETS2+CNN1, the median of 26 units (NLL difference to the additive baseline: +0.006 nats per UMI); Pearson correlation with the real log fold change on its top-20 DE genes: 0.91 for CAD and 0.98 for the reference.

(f) *K562, control cell as residual only, pair of two seen genes*: FOXA1+FOXL2, the median of 26 units (NLL difference to the additive baseline: −0.004 nats per UMI); Pearson correlation with the real log fold change on its top-20 DE genes: 0.87 for CAD and 0.83 for the reference.

(g) *PBMC, zero-shot cell type (fold 0)*: naive thymus-derived CD8-positive, alpha-beta T cell, the median of 4 units (NLL difference to the nearest Cell Ontology type: −0.728 nats per UMI).

Model	NLL ↓	cell NLL ↓	MMD ↓	AUC ↓	r_μ ↑	Type acc. ↑
frequency	**5.629**	**5.629**	0.1508	0.963	0.947	**0.887**
smoothed	5.801 (+0.172 [+0.144, +0.196])	5.801	0.2245	0.994	0.923	0.761
log1p	5.773 (+0.144 [+0.097, +0.199])	5.773	**0.0934**	**0.958**	**0.955**	0.854
sqrt	5.747 (+0.118 [+0.087, +0.152])	5.747	0.0939	0.962	0.955	0.835

Table 9: Human PBMC (Hao et al.), validation split: 19 held-out cell types; CAD with the Qwen3.5-0.8B backbone and four target transformations. NLL: multinomial negative log-likelihood per UMI of the pooled real cells under the model (Delta: paired difference to frequency [95% bootstrap over cell types]). MMD, AUC (real vs generated classifier; 0.5 is ideal), r_μ (gene-mean correlation), type acc.: a classifier trained on real cells recovers the requested type. The generation metrics use cells from the shared Dirichlet-multinomial sampler. Best in bold; our rows shaded.

E Transformed training targets (log1p, square root)

Table 9 (validation, 19 cell types, Qwen3.5-0.8B backbone) shows that the transformation trades likelihood for sample quality on PBMC. Frequency gives the best NLL, and smoothing is worse on every metric ($+0.172$ nats, 95% CI $+0.144$ to $+0.196$; MMD $+0.074$). Log1p and square-root targets are $+0.144$ and $+0.118$ nats worse in NLL, yet their MMD is lower (-0.057, CI -0.107 to -0.016, and -0.057, CI -0.109 to -0.014) with unchanged real-versus-generated AUC. A flatter target moves mass to rarely expressed genes, which lowers the likelihood of raw counts while placing generated cells closer to real ones in PCA space.

For perturbation response the trade-off does not appear (Gemma backbone, test, 76 perturbations; Appendix J). Relative to the frequency target, log1p is 0.288 nats worse in NLL (CI $+0.249$ to $+0.333$) and square root 0.250 worse (CI $+0.219$ to $+0.282$), and both also have a higher MMD (0.094 and 0.087 against 0.039) and a higher real-versus-generated AUC (0.875 and 0.871 against 0.811). We therefore keep frequency as the default, and we report the sample-quality advantage of flattened targets as specific to the conditional PBMC task in our experiments.

F UMAP views of generated cells

G Pre-registered decision rules and the test-split record

Beyond the main configurations, every variant was first run on validation (Qwen3.5-0.8B backbone, seed 0, matched control cells, comparators re-run with the same control cells) and promoted to the test split only if, on validation, its paired 95% interval against its comparator excluded zero in its favor on at least one metric fixed in advance, and no such metric was worse with an interval excluding zero; at most three new configurations were promoted. Promoted configurations were then run once per seed (0, 1, 2) on test with the Gemma backbone. The rules and decisions were written to the repository before the test runs.

Variant	Comparator	Primary metric(s)	Decision
Control residual, no control text	control residual with mass text	per-cell NLL	beat comparator; test, 3 seeds
Control text, no residual	control residual with mass text	per-cell NLL	worse; test, 1 seed (ablation)
Contrast control text	control residual with mass text	per-cell NLL, MMD, r_Δ	promoted
Gene summaries in the prompt	other gene's summary; opaque ids	NLL, r_Δ on unseen singles	not promoted
UMI-weighted loss	same model, equal weights	NLL, r_Δ, r_{DE}	not promoted
Negative-binomial head	softmax head	NLL, r_Δ, r_{DE}	promoted (Norman only)
Low-rank head	softmax head	per-cell NLL, MMD, AUC	promoted (no control cell)
Zero-shot cell types	nearest Cell Ontology type	NLL	passed (gate G3)
C2S-Gemma mixing weight	—	validation NLL	0.9 (grid maximum)

Table 10: Validation decisions taken before the test runs.

Four analyses were added after the test results had been seen, and are labeled as such where they appear: the R^2 of the rank reconstruction (Section 5.1), the control-cell residual without a language model (Section 5.4; a first version that divided by the pooled control frequency instead of the mean prior was not a fair analogue and was discarded), the

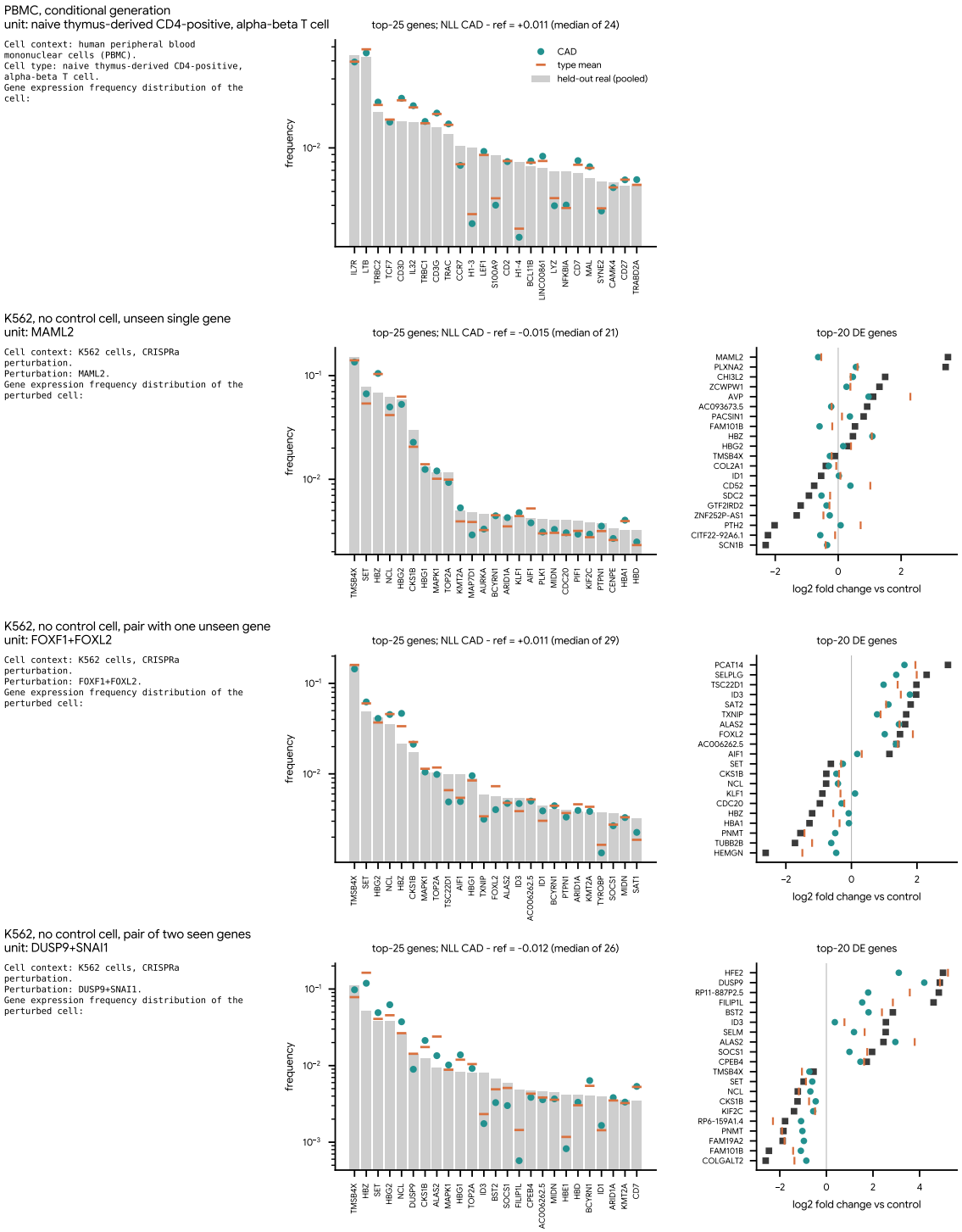


Figure 5: Prompts and decoded distributions, one median example per use case (test split). Left: the prompt given to CellAsDistribution. Middle: frequency of the 25 genes with the highest ground-truth frequency (log scale) for the held-out real cells (grey bars), CellAsDistribution (teal dots) and the reference (orange ticks). Right, perturbations only: log fold change over the held-out control mean on the unit's top-20 DE genes for real cells, CellAsDistribution and the reference. Shown here: conditional generation and the three strata without a control cell; the remaining cases are in Figure 6.

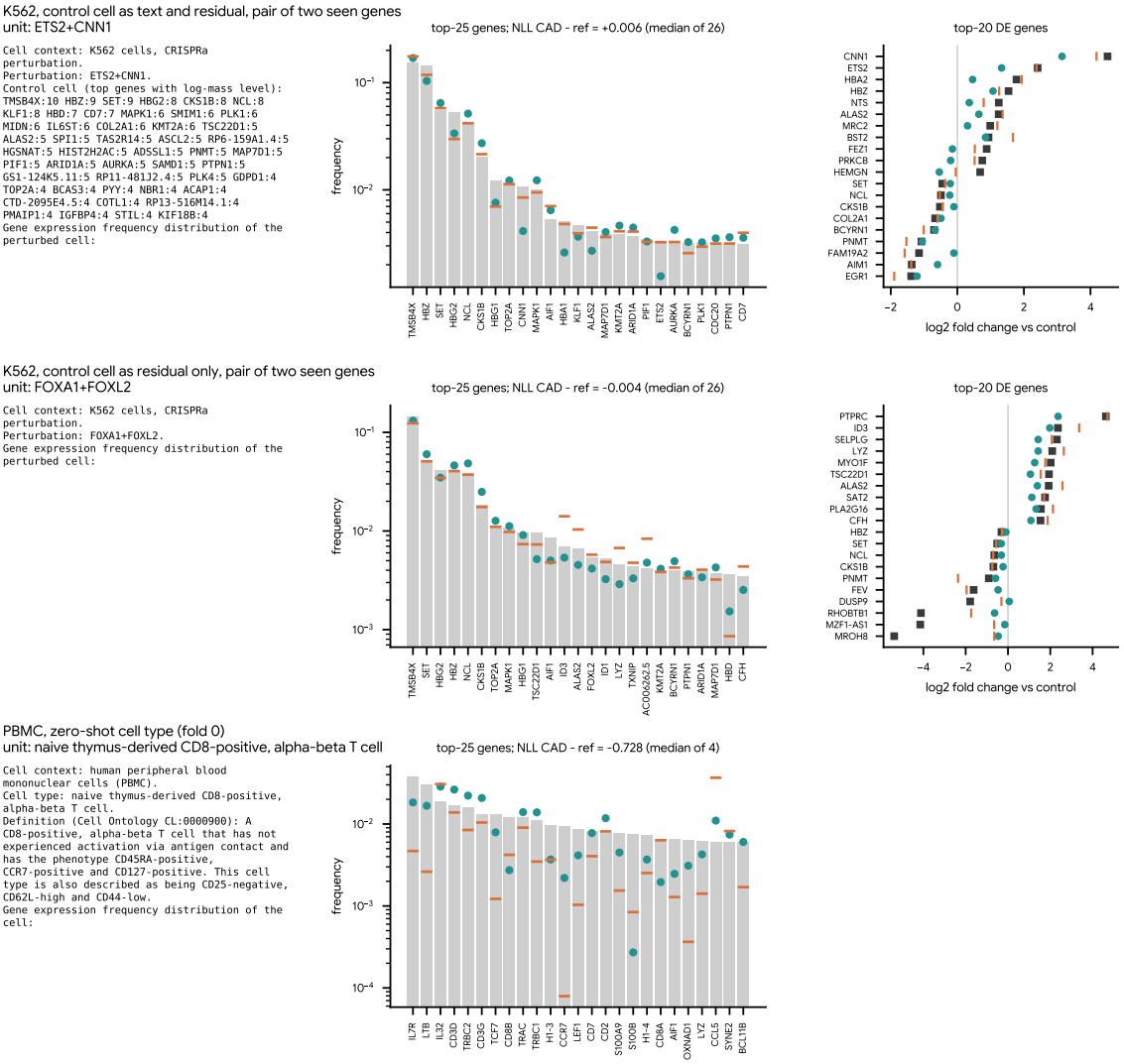


Figure 6: Prompts and decoded distributions (continued from Figure 5): perturbation response with the control cell as text and residual, and as residual only, and zero-shot cell types.

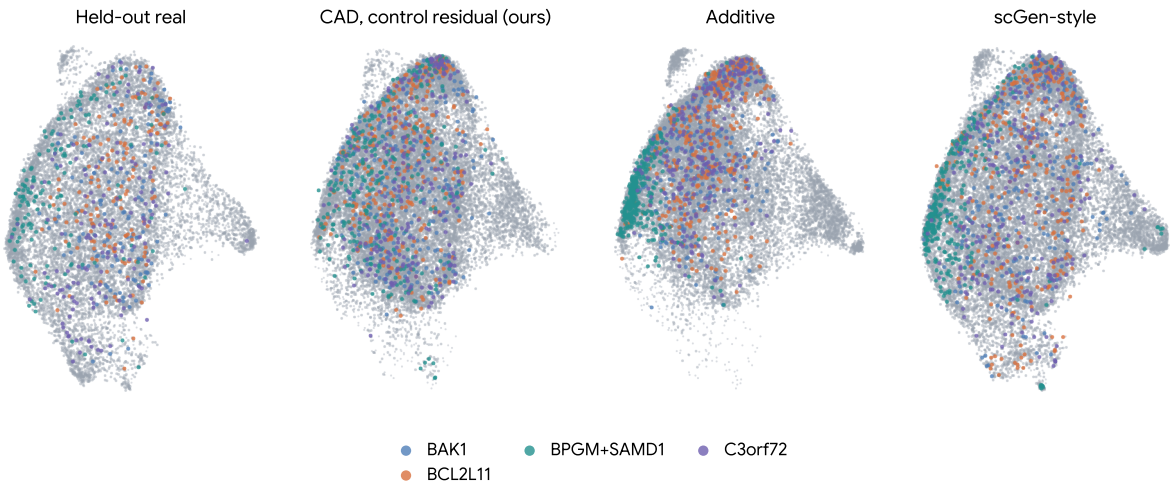


Figure 7: UMAP of held-out real perturbed K562 cells (test split) and of cells generated from the matched control cells by CellAsDistribution with the control-cell residual (Gemma, seed 0), by the additive baseline and by the scGen-style latent shift, all drawn with the shared multinomial sampler. PCA and UMAP are fit on real cells only, and generated cells are projected. Colors: the four most frequent held-out perturbations; grey: the others.

name-overlap and Cell Ontology parent baselines for zero-shot types (Section 5.5), and the decoding-speed benchmark (Section 5.2). Each can only weaken our claims.

Deviations, all disclosed: a 128-gene mass-level control text was not evaluated because it repeatedly ran out of GPU memory; three Qwen control-residual test runs ran out of memory during the test lanes and were re-run alone with the same configuration; and the zero-shot gate was applied on the 8 types of folds 0 and 1 because three validation runs of folds 2–4 had run out of memory, which were then re-run (on all 19 types the gate gives the same decision, -0.476 nats per UMI, CI -0.971 to -0.123).

Revision 3 (written before its new runs). Two controls for the language model (a frozen backbone with an MLP head, and a decoder without a language model; Section 5.2) were specified, with their comparators, metrics and decision rule, in `experiments/PREREGISTRATION_rev3.md` before they were run. As controls rather than candidate improvements, they were run on validation (Qwen, seed 0) and then on test (Gemma, seeds 0–2) whatever their validation result. Four further analyses of saved outputs were added after the test results had been seen: the downstream and target metrics with the perturbed genes excluded and the additive-plus-target-induction baseline (Section 5.3); the learned rank-to-frequency maps, including the re-decoding of the C2S-Gemma sentences (Section 5.1); and the co-expression comparison (Appendix I). They use no new training on test data. Without exclusion, the downstream code reproduces the earlier r_{DE} values exactly.

H Negative-binomial and low-rank heads

Negative-binomial head. The head adds a per-gene inverse dispersion $\theta_g(c)$, $\log \theta_g(c) = \beta_g + v_g^\top h$, and is trained on the negative-binomial likelihood of the counts with mean $N_i q_g$, so high-mass genes no longer dominate the gradient. On the Norman test split (no control cell, against the softmax head) it raises r_Δ by 0.020 (CI 0.012 to 0.029), and by 0.049 on unseen single genes (CI 0.034 to 0.067), at a small cost in pooled multinomial NLL ($+0.015$, CI $+0.006$ to $+0.028$). On PBMC it did not help on validation and was not promoted.

Low-rank logistic-normal head. The logits become $z = Wh + \log b + U\,\mathrm{diag}(a(h))\,\epsilon$ with $\epsilon \sim \mathcal{N}(0, I_{16})$, so $q(c, z)$ varies from cell to cell along 16 learned gene programs; training uses a variational bound with an encoder of the target cell, and generation draws ϵ from the prior. Without a control cell it lowers the cell NLL by 0.089 (CI 0.080 to 0.098) on Norman, reaching the scGen-style model (-0.009, CI -0.019 to $+0.002$), and on PBMC from 5.623 to 5.552. The pooled NLL of the mean over draws is worse ($+0.062$, CI $+0.040$ to $+0.087$) and r_Δ lower (-0.067). The

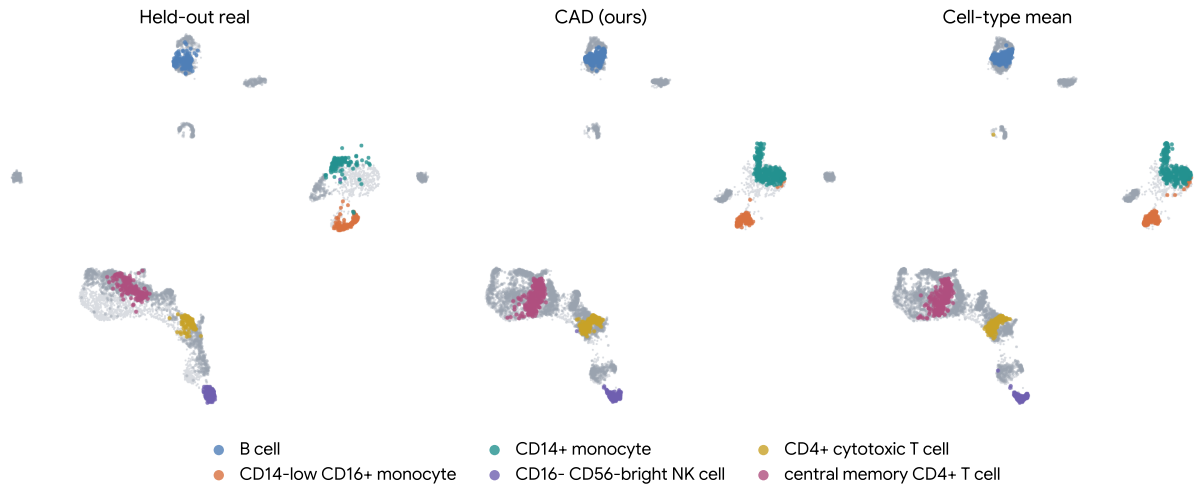


Figure 8: UMAP of held-out real PBMC cells (unseen donors, test split) and of cells generated by CellAsDistribution (Gemma, seed 0) and by the cell-type mean with the same sampler. PCA and UMAP are fit on real cells only, and generated cells are projected. Colors: the six most frequent cell types; grey: the others.

cell NLL is estimated from 50 (Norman) or 300 (PBMC) prior draws; this log-mean-exp estimator is biased upwards.

I Co-expression of generated cells

Generator	correlation error ↓	variance ratio	PC1-share error ↓
Additive	0.078	0.94	0.027
Additive, control residual (no LM)	0.078	1.01	0.027
CellAsDistribution, control residual	0.078	1.04	0.028
CellAsDistribution, low-rank head	0.074	1.04	0.021
CellAsDistribution, no control cell	0.079	0.99	0.026
Training-cell resampling	0.093	1.02	0.018
scGen-style	0.075	1.01	0.017

Table 11: Co-expression of generated cells, Norman test split (71 perturbations with at least 100 cells; 100 training-selected variable genes; seed 0; post hoc). Correlation error: mean absolute difference between gene–gene correlations across real and generated cells; variance ratio: total variance generated/real (1 is ideal).

For each test perturbation with at least 100 real cells, on the 100 panel genes with the largest variance of log-normalized expression in training cells, we compared the gene–gene Pearson correlations across real cells with those across the method's generated cells (multinomial sampler), and the total variance and the share of variance on the first principal component (Table 11; post hoc). With a median of about 216 panel UMIs per cell, correlations are dominated by counting noise, and every generator is within 0.005 of the others in correlation error. Resampling real cells of training perturbations is further from the held-out cells (0.093), so the generated cells reproduce perturbation-specific co-variation at least as well as real cells of other conditions. The low-rank head and the scGen-style shift, which model variation between cells explicitly, are slightly closer than the flat decoders, and the additive baseline under-disperses (variance ratio 0.94).

Model	NLL	ΔNLL to additive	cell NLL	MMD	AUC	r_Δ	r_{DE}
Control mean	5.352	+0.158 [+.108, +.215]	5.352	0.085	0.866	0.23	0.41
Perturbed mean	5.289	+0.094 [+.058, +.136]	5.289	0.073	0.903	0.12	0.58
Additive (reference)	5.195	0	5.195	0.052	0.855	0.44	0.77
Additive + target induction	5.194	−0.001 [−.003, +.000]	5.194	0.051	0.857	0.44	0.77
Ridge on gene embeddings	5.265	+0.070 [+.031, +.114]	5.265	0.057	0.849	0.46	0.68
scGen-style	5.201	+0.006 [−.014, +.022]	5.105	0.029	0.769	0.51	0.72
GEARS	5.755	+0.561 [+.518, +.603]	5.755	0.198	0.980	0.14	0.49
C2S, autoregressive	5.288	+0.094 [+.059, +.133]	5.279	0.075	0.906	0.10	0.56
Additive, control residual	5.199	+0.005 [−.001, +.011]	5.134	0.038	0.815	0.44	0.76
CellAsDistribution, no control cell	5.186	−0.009 [−.037, +.010]	5.186	0.039	0.812	0.50	0.77
CellAsDistribution, control residual	5.197	+0.002 [−.025, +.022]	5.129	0.030	0.785	0.46	0.72
CellAsDistribution, NB head	5.201	+0.006 [−.021, +.028]	5.201	0.036	0.797	0.52	0.77
CellAsDistribution, low-rank head	5.247	+0.053 [+.024, +.084]	5.096	0.040	0.791	0.43	0.73
CellAsDistribution, opaque gene ids	5.200	+0.005 [−.024, +.025]	5.200	0.048	0.845	0.43	0.69

Table 12: All methods, Norman test split (76 perturbations, seed 0, matched control cells). r_Δ: Pearson correlation of predicted and real change relative to the training perturbed centroid over all panel genes except the perturbed ones; r_{DE} as in Table 2. GEARS and control-mean rows: perturbed genes not excluded (earlier run).

Model	NLL ↓	cell NLL ↓	MMD ↓	AUC ↓	r_μ ↑
frequency	**5.186**	**5.186**	**0.0393**	**0.811**	**0.957**
log1p	5.474 (+0.288 [+0.249, +0.333])	5.474	0.0941	0.875	0.938
sqrt	5.435 (+0.250 [+0.219, +0.282])	5.435	0.0869	0.871	0.940

Table 13: Norman K562, test split: 76 held-out perturbations; CAD (Gemma), no control cell, with three target transformations. NLL: multinomial negative log-likelihood per UMI of the pooled real cells under the model (Delta: paired difference to frequency [95% bootstrap over perturbations]). cell NLL: per-cell mixture over M random control cells. MMD, AUC (real vs generated classifier; 0.5 is ideal), r_μ (gene-mean correlation). The generation metrics use cells from the shared Dirichlet-multinomial sampler. Best in bold; our rows shaded.

J Additional tables (all methods, validation, sensitivity)

The tables in this section and in Appendix E come from the earlier runs, in which the control cells were drawn separately for each method; the main-text tables use matched control cells. Table 13 (test) compares the target transformations for perturbation response with the Gemma backbone, and Table 14 (test) compares the two control-cell formats. Tables 15 and 16 repeat the comparison with the other generators on the validation split, on which all decisions were made; Table 17 shows the LoRA-rank sensitivity on Norman (validation).

K Proofs (Observation 1 and the hierarchical head)

Observation 1. (i) $x = Np$ recovers the counts from (N, p), and (N, p) is a function of x. Under $\mathrm{Mult}(N, q)$, $\log P(x) = \log \frac{N!}{\prod_g x_g!} + \sum_g x_g \log q_g = \text{const} + N \sum_g p_g \log q_g$, which by Gibbs' inequality is maximized over $q \in \Delta$ at $q = p$. (ii) For any x with at least two distinct nonzero counts, x and $2x$ have the same ordering and hence the same sentence, and so do x and any count vector obtained by a strictly increasing change of the counts. (iii) x and $2x$ have the same binarization. For $f(t) = \log(1+t)$ or $f(t) = \sqrt{t}$, the renormalized vector is $p'_g = f(x_g)/\sum_h f(x_h)$, and $p' = p$ requires $f(x_g)/x_g$ to be equal for all genes with $x_g > 0$; because $f(t)/t$ is strictly decreasing for $t > 0$, this holds only when all nonzero counts are equal.

Throughout, Δ is the simplex over the G panel genes and modules $S_1, \ldots, S_M$ partition the genes; P1, P3 and P4 concern the hierarchical head of Appendix L, and P2 concerns the NLL used throughout.

Model	NLL ↓	cell NLL ↓	MMD ↓	AUC ↓	r_μ ↑
mass text	5.199	5.173	**0.0371**	0.816	**0.955**
rank text	**5.192 (-0.007 [-0.012, +0.000])**	**5.159**	0.0375	**0.809**	0.954

Table 14: Norman K562, test split: 76 held-out perturbations; CAD (Gemma), control residual, with the control cell given as mass levels or as a 200-gene rank sentence. NLL: multinomial negative log-likelihood per UMI of the pooled real cells under the model (Delta: paired difference to mass text [95% bootstrap over perturbations]). cell NLL: per-cell mixture over M random control cells. MMD, AUC (real vs generated classifier; 0.5 is ideal), r_μ (gene-mean correlation). The generation metrics use cells from the shared Dirichlet-multinomial sampler. Best in bold; our rows shaded.

Model	NLL ↓	cell NLL ↓	MMD ↓	AUC ↓	r_μ ↑	Type acc. ↑
Global mean	6.700 (+1.073 [+0.892, +1.283])	6.700	0.8462	1.000	0.565	0.048
Cell-type mean	**5.627**	**5.627**	0.1688	0.983	0.942	**0.928**
scVI	5.634 (+0.007 [-0.005, +0.018])	5.634	0.1576	0.968	0.946	0.906
scANVI	5.630 (+0.003 [-0.007, +0.012])	5.630	0.1494	0.967	0.947	0.878
CAD (Gemma)	5.628 (+0.001 [-0.007, +0.008])	5.628	**0.1440**	0.972	**0.948**	0.887
CAD (Qwen), seed 1	5.632 (+0.005 [-0.003, +0.011])	5.632	0.1464	**0.963**	0.947	0.900

Table 15: Human PBMC (Hao et al.), validation split: 19 held-out cell types; CAD rows: first seed unless stated; baselines: one run each. NLL: multinomial negative log-likelihood per UMI of the pooled real cells under the model (Delta: paired difference to Cell-type mean [95% bootstrap over cell types]). MMD, AUC (real vs generated classifier; 0.5 is ideal), r_μ (gene-mean correlation), type acc.: a classifier trained on real cells recovers the requested type. The generation metrics use cells from the shared Dirichlet-multinomial sampler. Best in bold; our rows shaded.

P1 (normalization and exact factorization). For any $q^{\mathrm{root}} \in \Delta^M$ and $q^m \in \Delta^{|S_m|}$, $q(g) = q^{\mathrm{root}}(m(g))\, q^{m(g)}(g)$ lies in Δ, and for counts X with module totals n_m,

$$-\sum_g X_g \log q(g) = -\sum_m n_m \log q^{\mathrm{root}}(m) - \sum_m \sum_{g \in S_m} X_g \log q^m(g).$$

Every q in the interior of Δ factors uniquely this way, so the hierarchical head is a *re-parametrization* of the flat head: it changes what the network must output, not the family of distributions it can represent.

P2 (the metric is proper). If the counts X of a perturbation's cells satisfy $\mathbb{E}[X_g / \sum_h X_h] = \pi_g$ given the total, the expected NLL per UMI of q is $H(\pi) + \mathrm{KL}(\pi \,\|\, q)$ and is uniquely minimized at $q = \pi$. Differences in expected NLL between two decoders therefore equal differences in $\mathrm{KL}(\pi \| q)$.

P3 (tilt identity and its population minimizer). For $f, \tilde{p}$ in the interior of Δ, $q_g \propto f_g t_g$ equals $\tilde{p}_g$ exactly. If a context is paired with bags b having tilts t_b, the minimizer of the expected soft cross-entropy is $\mathbb{E}[t_b]$, so the implied decoder is $q_g \propto \mathbb{E}_b[\tilde{p}_{b,g}/Z_b]$: a $1/Z_b$-weighted average of bag frequencies, which equals the plain average whenever Z_b is constant within a context. The target is the shrunk $\tilde{p}$, so $\|\tilde{p} - p\|_1 \le \frac{\lambda}{n+\lambda}\|f - p\|_1 \le 2\frac{\lambda}{n+\lambda}$ ($\approx 1\%$ for $n \approx 2\times10^5$ UMIs per bag).

P4 (sampled-leaf loss). Let $T_m = \sum_{g \in S_m} t_g$. With modules $A = (m_1, \ldots, m_R)$ drawn i.i.d. from T, $\mathrm{CE}(T, r^{\mathrm{root}}) + \mathbb{E}\big[\frac{1}{R}\sum_r \mathrm{CE}(t|_{m_r}, r^{m_r})\big] = \mathrm{CE}(t, r)$, the full tree cross-entropy. *Our implementation samples without replacement and the trainer averages questions with equal weight, so its record loss is a weighted surrogate* ($1/(R{+}1)$ per question), not this unbiased estimator; the two share their minimizer only when every conditional is exactly representable (the well-specified regime).

P1. $\sum_g q(g) = \sum_m q^{\mathrm{root}}(m) \sum_{g \in S_m} q^m(g) = \sum_m q^{\mathrm{root}}(m) = 1$ and every factor is nonnegative. The loss identity follows from $\log q(g) = \log q^{\mathrm{root}}(m(g)) + \log q^{m(g)}(g)$ and $\sum_{g \in S_m} X_g = n_m$. Conversely, for q in the

Model	NLL ↓	cell NLL ↓	MMD ↓	AUC ↓	r_μ ↑
Perturbed mean	5.314	5.314	0.0779	0.915	0.920
Additive	5.248 (-0.066 [-0.096, -0.040])	5.248	0.0459	0.838	0.952
Ridge	5.306 (-0.008 [-0.021, +0.004])	5.306	0.0670	0.874	0.936
scGen-style	5.271 (-0.043 [-0.072, -0.017])	**5.162**	**0.0300**	**0.774**	0.955
CAD (Gemma), no control cell	**5.235 (-0.079 [-0.105, -0.059])**	5.235	0.0360	0.799	**0.959**
CAD (Gemma), control residual, mass text	5.246 (-0.068 [-0.094, -0.046])	5.221	0.0393	0.824	0.952
CAD (Gemma), control residual, rank text	5.240 (-0.075 [-0.099, -0.054])	5.209	0.0370	0.821	0.954
CAD (Qwen), no control cell, seed 1	5.239 (-0.076 [-0.102, -0.055])	5.239	0.0400	0.823	0.955

Table 16: Norman K562, validation split: 48 held-out perturbations; CAD rows: first seed unless stated; baselines: one run each. NLL: multinomial negative log-likelihood per UMI of the pooled real cells under the model (Delta: paired difference to Perturbed mean [95% bootstrap over perturbations]). cell NLL: per-cell mixture over M random control cells. MMD, AUC (real vs generated classifier; 0.5 is ideal), r_μ (gene-mean correlation). The generation metrics use cells from the shared Dirichlet-multinomial sampler. Best in bold; our rows shaded.

Model	NLL ↓	cell NLL ↓	MMD ↓	AUC ↓	r_μ ↑
rank 16	**5.235**	**5.235**	**0.0360**	**0.799**	**0.959**
rank 8	5.237 (+0.002 [-0.002, +0.007])	5.237	0.0365	0.812	0.959
rank 64	5.237 (+0.002 [-0.002, +0.007])	5.237	0.0386	0.809	0.957

Table 17: Norman K562, validation split: 48 held-out perturbations; CAD (Gemma), no control cell, with LoRA rank 16, 8 and 64. NLL: multinomial negative log-likelihood per UMI of the pooled real cells under the model (Delta: paired difference to rank 16 [95% bootstrap over perturbations]). cell NLL: per-cell mixture over M random control cells. MMD, AUC (real vs generated classifier; 0.5 is ideal), r_μ (gene-mean correlation). The generation metrics use cells from the shared Dirichlet-multinomial sampler. Best in bold; our rows shaded.

interior set $q^{\text{root}}(m) = \sum_{g \in S_m} q(g)$ and $q^m(g) = q(g)/q^{\text{root}}(m)$; these are the only factors that reproduce q because $q^{\text{root}}(m) = \sum_{g \in S_m} q(g)$ is forced by summing.

P2. Conditional on the total N, $\mathbb{E}[-\sum_g X_g \log q_g / N] = -\sum_g \pi_g \log q_g = H(\pi) + \mathrm{KL}(\pi \| q)$, and $\mathrm{KL} \geq 0$ with equality if and only if $q = \pi$ (Gibbs' inequality). Two decoders therefore differ in expected NLL by $\mathrm{KL}(\pi \| q_A) - \mathrm{KL}(\pi \| q_B)$. Per-UMI scoring weights each perturbation's cells by their UMIs; the paired bootstrap treats the perturbation as the unit.

P3. $f_g t_g = \tilde{p}_g / Z$, so $\sum_h f_h t_h = 1/Z$ and $f_g t_g / \sum_h f_h t_h = \tilde{p}_g$. For targets t_b drawn with context c,

$$\mathbb{E}_b[\mathrm{CE}(t_b, r)] = H(\mathbb{E} t_b) + \mathrm{KL}(\mathbb{E} t_b \| r) + \text{const}$$

is minimized by $r = \mathbb{E}[t_b]$, so $q_g \propto f_g \mathbb{E}[t_{b,g}] = \mathbb{E}[\tilde{p}_{b,g}/Z_b]$, a $1/Z_b$-weighted mean of $\tilde{p}_b$. With $\tilde{p} = (1-w)p + wf$ and $w = \lambda/(n+\lambda)$, $\|\tilde{p} - p\|_1 = w\|f - p\|_1 \leq 2w$.

P4. The chain rule gives $\mathrm{CE}(t, r) = \mathrm{CE}(T, r^{\text{root}}) + \sum_m T_m \, \mathrm{CE}(t|_m, r^m)$ with $t|_m = t_{S_m}/T_m$. For $m_1, \ldots, m_R$ i.i.d. from T, $\mathbb{E}[\mathrm{CE}(t|_{m_r}, r^{m_r})] = \sum_m T_m \mathrm{CE}(t|_m, r^m)$ for each r, which gives the claim. Sampling without replacement changes the inclusion probabilities away from T_m, and averaging over the $R{+}1$ answered questions gives each question weight $1/(R{+}1)$ instead of $(1, \frac{1}{R}, \ldots, \frac{1}{R})$; if every conditional is representable, each term is minimized at the true conditional and the minimizer is unchanged.

L Hierarchical decision head (ablation)

The hierarchical variant (CAD-N/CAD-R) answers one 64-option module question and four 32-option gene questions through the Unsloth decision head, trained on a *tilt* target (the ratio of the perturbed to the control frequency, renormalized, with the control prior f multiplied back at generation). On the corrected Norman split, trained on absolute

frequencies it is $+0.161$ [$+0.150$, $+0.174$] nats worse than the perturbed mean (control-state NLL 5.490 against 5.295 for the control frequencies); the tilt repairs the fit (control 5.300; tilt minus no-tilt -0.138 [-0.157, -0.119]) but the tilted head remains $+0.023$ [$+0.009$, $+0.038$] worse than the perturbed mean and 0.100 nats worse than the flat head. The biological partition beats a random one by 0.019 ($[-0.035, -0.002]$). We therefore use the flat head in the main text.